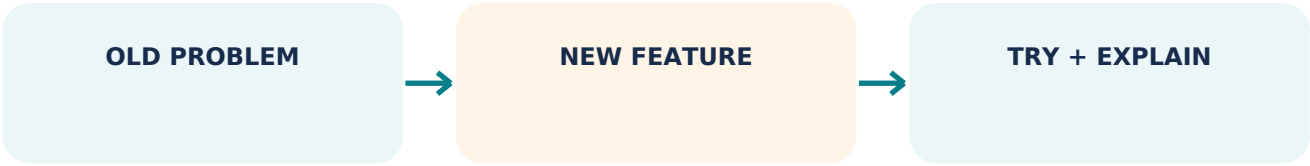
JAVA 8 → JAVA 21

The visual, code-first learning journey

Every Java release from 9 through 21 • small lessons • diagrams • code you can run • prediction questions

HOW TO USE THIS WORKBOOK

Read one lesson. First guess what the old code does. Then compare it with the new code. Stop at each orange question, predict the answer, and only then read the explanation. Try changing one line and run it in JDK 21.



The route we will follow

9–11	Collections, modules, var, String and HTTP APIs
12–15	Switch expressions and readable multiline strings
16–17	Records, pattern matching, sealed classes
18–20	Encoding defaults and preview-feature evolution
21	Virtual threads, switch patterns, record patterns, sequenced collections

Important: “Introduced” and “finalized” are not always the same. Preview features are experiments; the final syntax may arrive in a later release. All snippets are intended for JDK 21 unless marked as historical preview syntax.

JAVA 9 / VISUAL LESSON

Collection factories: List.of / Set.of / Map.of

01 WHY WOULD WE NEED THIS?

You have fixed options such as payment methods. You want callers to read them, but not add or replace entries.



02 BEFORE: HOW WE DID IT

```
List<String> methods = Collections.unmodifiableList(
    Arrays.asList("UPI", "CARD"));
```

03 AFTER: THE NEW WAY

```
List<String> methods = List.of("UPI", "CARD");
Set<String> codes = Set.of("IN", "US");
Map<String, Integer> limits = Map.of("UPI", 5);
```

WHAT IMPROVED?

Much shorter. The returned collections are unmodifiable and are not backed by a caller-owned mutable array.

WATCH OUT

Nulls are rejected. Set.of rejects duplicate elements; Map.of rejects duplicate keys. Elements themselves can still be mutable.

PAUSE & PREDICT

What happens? List x = List.of("A", "B"); x.set(0, "C");

Check your thinking: UnsupportedOperationException: unmodifiable means no set, add, or remove.

REMEMBER THIS

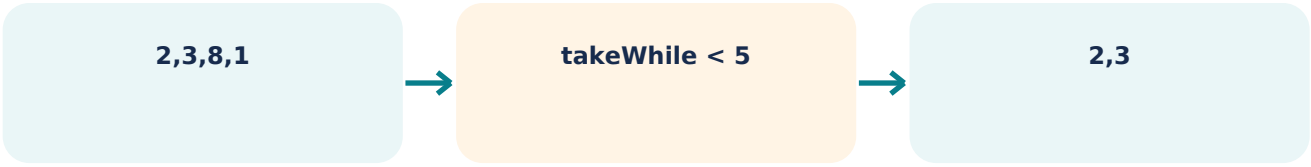
final stops reference reassignment, not object mutation. List.of protects the collection, not the state of mutable elements.

JAVA 9 / VISUAL LESSON

Streams: takeWhile and dropWhile

01 WHY WOULD WE NEED THIS?

You have ordered events and want to stop taking values at the first value that fails a rule.



02 BEFORE: HOW WE DID IT

```
List<Integer> first = nums.stream()
    .filter(n -> n < 5).collect(Collectors.toList());
```

03 AFTER: THE NEW WAY

```
List<Integer> first = nums.stream()
    .takeWhile(n -> n < 5).collect(Collectors.toList());
```

WHAT IMPROVED?

takeWhile takes the longest matching prefix; dropWhile skips that prefix. Unlike filter, these operations care about encounter order.

WATCH OUT

For an unordered stream, results are less intuitive. takeWhile is NOT the same as filter.

PAUSE & PREDICT

For [2, 3, 8, 1], what does takeWhile(n -> n < 5) return?

Check your thinking: [2, 3]. It stops when it reaches 8, so 1 is never taken.

REMEMBER THIS

Use filter for all matches; takeWhile for a matching prefix.

JAVA 9 / VISUAL LESSON

Modules: module-info.java

01 WHY WOULD WE NEED THIS?

A large app has many packages. You want to state what it depends on and which packages other modules may use.



02 BEFORE: HOW WE DID IT

```
// Java 8: dependencies usually on the classpath
// Public classes may be reachable by other code.
```

03 AFTER: THE NEW WAY

```
module com.example.payments {
    requires java.sql;
    exports com.example.payments.api;
}
```

WHAT IMPROVED?

JPMS adds explicit module dependencies and package exports. It can improve encapsulation and dependency clarity.

WATCH OUT

You do not need to modularize every app. Reflection and older libraries can need extra configuration.

PAUSE & PREDICT

Does public class mean every module can access it?

Check your thinking: No. In a named module, its package generally must also be exported.

REMEMBER THIS

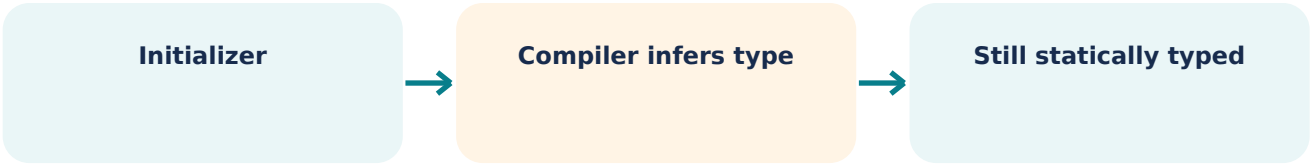
requires = dependency; exports = accessible package. Java 9 also added JShell and private interface methods.

JAVA 10 / VISUAL LESSON

var: less typing, same static typing

01 WHY WOULD WE NEED THIS?

You have a long local variable type that the compiler can clearly infer.



02 BEFORE: HOW WE DID IT

```
Map<String, List<Integer>> scores = new HashMap<>();
```

03 AFTER: THE NEW WAY

```
var scores = new HashMap<String, List<Integer>>();
// Compiler still knows the exact type.
```

WHAT IMPROVED?

Less repetition while keeping compile-time type checking.

WATCH OUT

Only for local variables and certain other local contexts; not fields or method return types. Avoid vague names and unclear initializers.

PAUSE & PREDICT

Will this compile? var x = null;

Check your thinking: No. The compiler cannot infer a type from null alone.

REMEMBER THIS

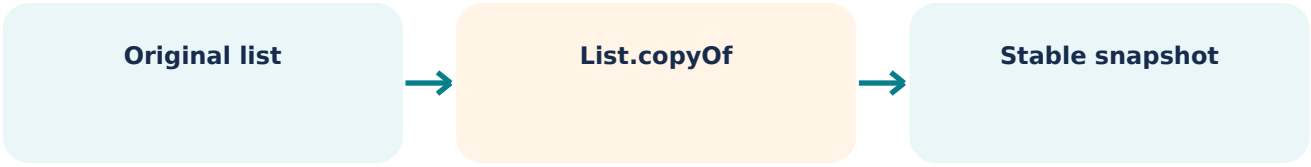
var is NOT dynamic typing. Use it when the type remains obvious.

JAVA 10 / VISUAL LESSON

List.copyOf: a frozen snapshot

01 WHY WOULD WE NEED THIS?

You want a caller to see the list as it looked at one moment, not later additions to the original.



02 BEFORE: HOW WE DID IT

```
List<String> snapshot = Collections.unmodifiableList(
    new ArrayList<>(original));
```

03 AFTER: THE NEW WAY

```
List<String> snapshot = List.copyOf(original);
original.add("NEW");
System.out.println(snapshot);
```

WHAT IMPROVED?

One concise call makes an unmodifiable snapshot of element references.

WATCH OUT

Nulls are rejected. Mutable elements are shared. If input is already a suitable unmodifiable list, the implementation may reuse it.

PAUSE & PREDICT

Original starts [A]. After copyOf, original.add(B). What does snapshot show?

Check your thinking: [A]. The copied list does not track later structural changes.

REMEMBER THIS

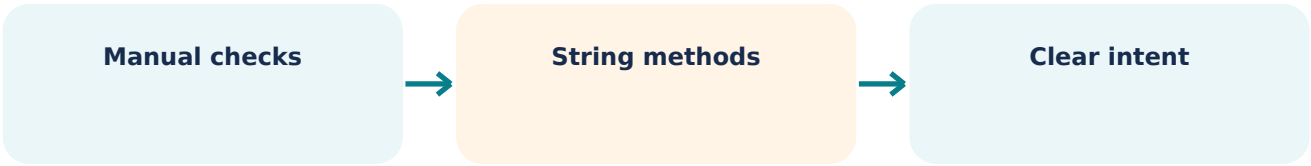
unmodifiableList(original) = live view; copyOf(original) = snapshot of element references.

JAVA 11 / VISUAL LESSON

String helpers: less manual checking

01 WHY WOULD WE NEED THIS?

You want to tell whether user input is blank, remove surrounding whitespace, or repeat a separator.



02 BEFORE: HOW WE DID IT

```
boolean blank = input.trim().isEmpty();
String cleaned = input.trim();
```

03 AFTER: THE NEW WAY

```
boolean blank = input.isBlank();
String cleaned = input.strip();
String bar = "-".repeat(5);
```

WHAT IMPROVED?

isBlank handles Unicode whitespace better than trim().isEmpty() in many cases; strip is Unicode-aware.

WATCH OUT

strip and trim are not identical. These methods do not change the original String.

PAUSE & PREDICT

Does " ".isBlank() return true?

Check your thinking: Yes. An empty String is also blank.

REMEMBER THIS

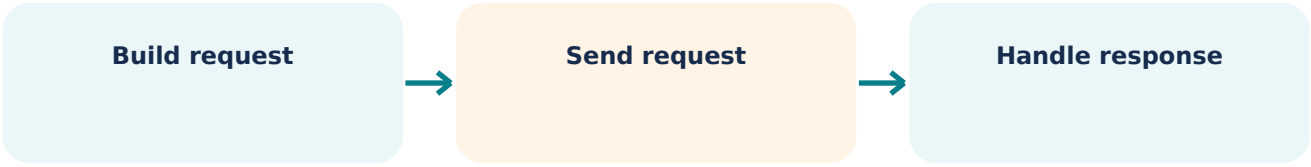
Java 11 also added lines(), Files.readString/writeString, Optional.isEmpty, and the standard HTTP Client.

JAVA 11 / VISUAL LESSON

HTTP Client: standard HTTP calls

01 WHY WOULD WE NEED THIS?

Your app calls another service. Older JDK APIs could be cumbersome for modern HTTP use.



02 BEFORE: HOW WE DID IT

```
URL url = new URL("https://example.com");
HttpURLConnection c = (HttpURLConnection) url.openConnection();
```

03 AFTER: THE NEW WAY

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder(
    URI.create("https://example.com")).build();
HttpResponse<String> response = client.send(
    request, HttpResponse.BodyHandlers.ofString());
```

WHAT IMPROVED?

A standard client with a fluent API, async support, and HTTP/2 support.

WATCH OUT

send() blocks the caller. Handle IOException and InterruptedException; use sendAsync for async calls.

PAUSE & PREDICT

Does HttpClient.newHttpClient() send a request immediately?

Check your thinking: No. It creates a client; send or sendAsync performs the request.

REMEMBER THIS

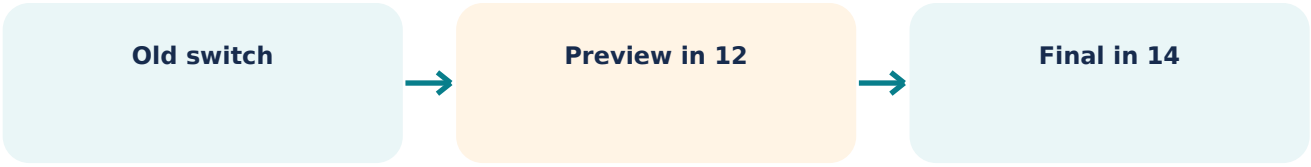
Java 11 finalized the HTTP Client API (first incubated in Java 9).

JAVA 12 / VISUAL LESSON

Switch expressions begin (preview)

01 WHY WOULD WE NEED THIS?

Traditional switch statements need repeated assignments and break statements.



02 BEFORE: HOW WE DID IT

```
String label;
switch (status) {
 case 1: label = "OK"; break;
 default: label = "OTHER";
}
```

03 AFTER: THE NEW WAY

```
// Modern final form (Java 14)
String label = switch (status) {
 case 1 -> "OK";
 default -> "OTHER";
};
```

WHAT IMPROVED?

The preview started in Java 12. The final feature arrived in Java 14.

WATCH OUT

Java 12 preview syntax is historical. Learn the finalized Java 14 form for normal Java 21 code.

PAUSE & PREDICT

Does arrow-style switch fall through to the next case?

Check your thinking: No. Arrow rules do not fall through.

REMEMBER THIS

Preview is a trial stage, not the same as final release.

JAVA 13 / VISUAL LESSON

Text blocks begin (preview)

01 WHY WOULD WE NEED THIS?

You have SQL or JSON spanning many lines. Escaped quotes and newline strings are hard to read.



02 BEFORE: HOW WE DID IT

```
String sql = "SELECT id, name\n" +
             "FROM users\n" +
             "WHERE active = true";
```

03 AFTER: THE NEW WAY

```
// Final form from Java 15
String sql = """
    SELECT id, name
    FROM users
    WHERE active = true
    """;
```

WHAT IMPROVED?

The Java 13 preview began the move to readable multiline text. Finalized in Java 15.

WATCH OUT

Indentation and trailing newline rules matter. Text blocks are still ordinary Strings.

PAUSE & PREDICT

Is a text block a new mutable string type?

Check your thinking: No. It is a String literal and Strings are immutable.

REMEMBER THIS

Java 13 introduced text blocks as a preview, not a final feature.

JAVA 14 / VISUAL LESSON

Switch expressions become final

01 WHY WOULD WE NEED THIS?

You want a switch that returns one value and makes missing cases easier to notice.



02 BEFORE: HOW WE DID IT

```
int fee;
switch (tier) {
 case "VIP": fee = 0; break;
 default: fee = 20;
}
```

03 AFTER: THE NEW WAY

```
int fee = switch (tier) {
 case "VIP" -> 0;
 default -> 20;
};
```

WHAT IMPROVED?

Cleaner expressions, no accidental fall-through with arrow cases, and a value is produced.

WATCH OUT

Switch expressions must be exhaustive. Use yield for a block that computes a value.

PAUSE & PREDICT

Can you omit default when switching on String and only handle VIP?

Check your thinking: Not for a switch expression: it must cover all possible values.

REMEMBER THIS

Java 14 also previewed records and pattern matching for instanceof.

JAVA 15 / VISUAL LESSON

Text blocks become final

01 WHY WOULD WE NEED THIS?

You need to embed a readable multiline JSON payload in Java.



02 BEFORE: HOW WE DID IT

```
String json = "{\n" +
 "  \"ok\": true\n" +
 "}";
```

03 AFTER: THE NEW WAY

```
String json = """
    {
      "ok": true
    }
    """;
```

WHAT IMPROVED?

Less escaping, much easier to review.

WATCH OUT

Whitespace is significant; compare exact strings when formatting matters.

PAUSE & PREDICT

Can you call json.length() on a text block?

Check your thinking: Yes. It produces an ordinary String.

REMEMBER THIS

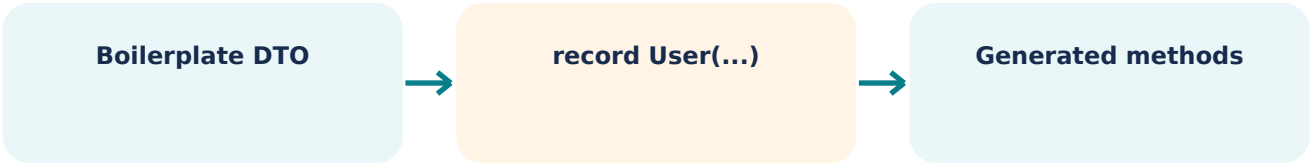
Text blocks were finalized in Java 15.

JAVA 16 / VISUAL LESSON

Records: small data carriers

01 WHY WOULD WE NEED THIS?

You keep writing constructors, accessors, equals, hashCode, and toString for simple DTOs.



02 BEFORE: HOW WE DID IT

```
final class User {
 private final String name;
 User(String name) { this.name = name; }
 String name() { return name; }
}
```

03 AFTER: THE NEW WAY

```
record User(String name) {}
User u = new User("Asha");
System.out.println(u.name());
```

WHAT IMPROVED?

Records generate the standard data-carrier methods for you.

WATCH OUT

Records are shallowly immutable: a record containing a mutable List can still expose that list. Records cannot extend another class.

PAUSE & PREDICT

Is a record field reassigned after construction?

Check your thinking: No, record component fields are final. The referenced object may still be mutable.

REMEMBER THIS

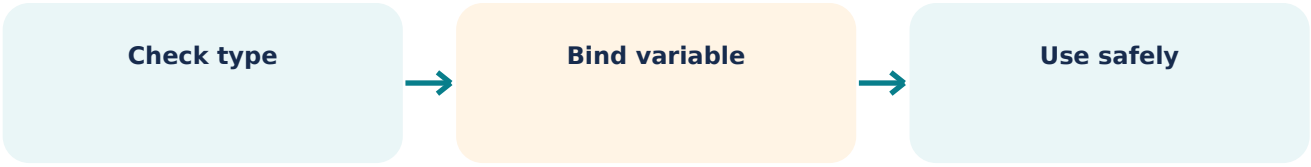
Java 16 finalized records and instanceof pattern matching.

JAVA 16 / VISUAL LESSON

Pattern matching for instanceof

01 WHY WOULD WE NEED THIS?

You test a type and then immediately cast it. The extra cast is repetitive.



02 BEFORE: HOW WE DID IT

```
if (obj instanceof String) {
 String s = (String) obj;
 System.out.println(s.length());
}
```

03 AFTER: THE NEW WAY

```
if (obj instanceof String s) {
    System.out.println(s.length());
}
```

WHAT IMPROVED?

Checks the type and creates a correctly typed variable in one step.

WATCH OUT

The pattern variable is only in scope where the compiler knows the match succeeded.

PAUSE & PREDICT

Does this work if obj is null?

Check your thinking: No match; the if body is skipped.

REMEMBER THIS

The variable s is not a copy of the String object.

JAVA 17 / VISUAL LESSON

Sealed classes: control the family

01 WHY WOULD WE NEED THIS?

You want a payment result to be only Success or Failure, not any arbitrary implementation.



02 BEFORE: HOW WE DID IT

```
interface Result {}
class Success implements Result {}
class Failure implements Result {}
```

03 AFTER: THE NEW WAY

```
sealed interface Result permits Success, Failure {}
final class Success implements Result {}
final class Failure implements Result {}
```

WHAT IMPROVED?

You can restrict permitted subclasses and model a closed set of cases.

WATCH OUT

Permitted subclasses must follow the sealed hierarchy rules: final, sealed, or non-sealed.

PAUSE & PREDICT

Can a random third class implement Result?

Check your thinking: No, not unless permitted by the sealed declaration.

REMEMBER THIS

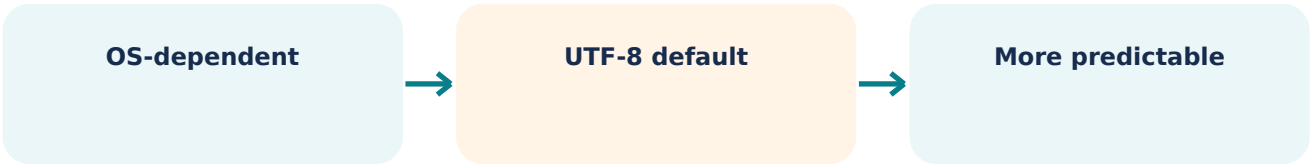
Java 17 is LTS. Strong JDK encapsulation also became stricter.

JAVA 18 / VISUAL LESSON

UTF-8 becomes the default charset

01 WHY WOULD WE NEED THIS?

Text behaves differently on two machines because each machine uses a different default encoding.



02 BEFORE: HOW WE DID IT

```
// Before Java 18: default could vary by OS
String text = new String(bytes);
```

03 AFTER: THE NEW WAY

```
// Java 18+: UTF-8 is the standard default
String text = new String(bytes);
// Better for protocols: specify charset explicitly
String safe = new String(bytes, StandardCharsets.UTF_8);
```

WHAT IMPROVED?

More consistent default text handling across platforms.

WATCH OUT

Existing apps depending on a legacy default charset can behave differently after upgrade.

PAUSE & PREDICT

Should you omit the charset for a network protocol?

Check your thinking: Prefer specifying UTF-8 explicitly: it documents the contract.

REMEMBER THIS

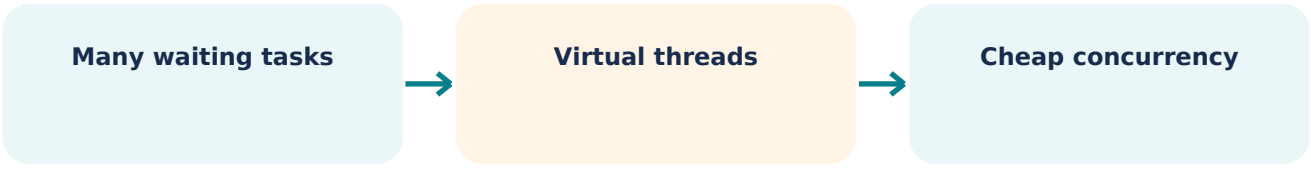
Java 18 also introduced the Simple Web Server tool.

JAVA 19 / VISUAL LESSON

Virtual threads arrive as preview

01 WHY WOULD WE NEED THIS?

A server has many requests waiting on network or database I/O. Platform threads are relatively expensive at very large counts.



02 BEFORE: HOW WE DID IT

```
ExecutorService pool = Executors.newFixedThreadPool(50);
pool.submit(() -> loadFromDatabase());
```

03 AFTER: THE NEW WAY

```
// Java 19 preview; final Java 21 API shown
Thread.startVirtualThread(() -> loadFromDatabase());
```

WHAT IMPROVED?

Virtual threads make thread-per-task style practical for many blocking I/O workloads.

WATCH OUT

Preview in Java 19. They do not speed up CPU-bound work by magic.

PAUSE & PREDICT

Would 100 CPU-heavy tasks finish faster just because threads are virtual?

Check your thinking: Not necessarily. CPU cores still limit parallel computation.

REMEMBER THIS

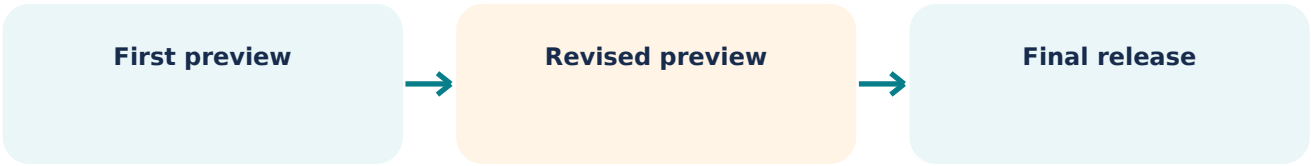
Java 19 previewed virtual threads and record patterns.

JAVA 20 / VISUAL LESSON

Preview features evolve

01 WHY WOULD WE NEED THIS?

A feature may change after real developer feedback. You should not treat a preview as permanent syntax.



02 BEFORE: HOW WE DID IT

```
// Java 19 preview experiments
// may need --enable-preview
```

03 AFTER: THE NEW WAY

```
// Java 20: second previews of virtual threads
// and record patterns; learn final Java 21 forms.
```

WHAT IMPROVED?

Preview cycles let Java refine language and runtime design before finalizing.

WATCH OUT

Production projects usually prefer finalized features. Preview code may require flags and migration.

PAUSE & PREDICT

Was Java 20 virtual threads the final release?

Check your thinking: No. Virtual threads were finalized in Java 21.

REMEMBER THIS

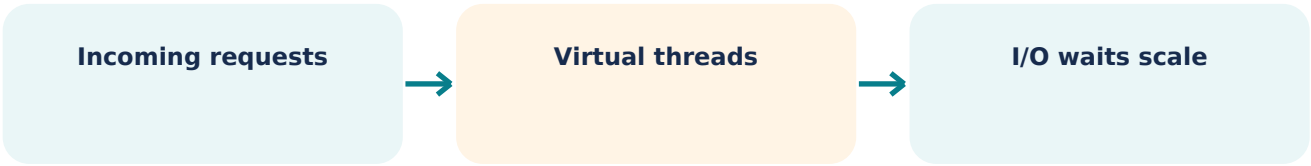
Java 20 also incubated scoped values and structured concurrency; neither was final in Java 21.

JAVA 21 / VISUAL LESSON

Virtual threads become final

01 WHY WOULD WE NEED THIS?

A backend spends much of its time waiting for I/O. You want simple blocking code without a huge pool of expensive platform threads.



02 BEFORE: HOW WE DID IT

```
ExecutorService pool = Executors.newFixedThreadPool(100);
pool.submit(() -> fetchOrder());
```

03 AFTER: THE NEW WAY

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    executor.submit(() -> fetchOrder());
}
```

WHAT IMPROVED?

One virtual thread per task works well for large numbers of mostly blocking I/O tasks.

WATCH OUT

Do not pool virtual threads. Use separate limits for scarce resources (like DB connections). Watch for pinning and ThreadLocal usage.

PAUSE & PREDICT

Is one virtual thread per request the same as one OS thread per request?

Check your thinking: No. Many virtual threads can be scheduled on fewer carrier platform threads.

REMEMBER THIS

Virtual threads are final in Java 21, not a preview.

JAVA 21 / VISUAL LESSON

Pattern matching for switch

01 WHY WOULD WE NEED THIS?

You have an Object that may be a String, Integer, or something else. Old code chains instanceof and casts.



02 BEFORE: HOW WE DID IT

```
if (value instanceof String) {
 String s = (String) value;
 System.out.println(s.length());
} else if (value instanceof Integer) {
 Integer n = (Integer) value;
 System.out.println(n + 1);
}
```

03 AFTER: THE NEW WAY

```
String result = switch (value) {
 case String s -> "len=" + s.length();
 case Integer n -> "next=" + (n + 1);
 case null -> "missing";
 default -> "other";
};
```

WHAT IMPROVED?

Clear, type-safe branching; patterns bind variables without manual casts.

WATCH OUT

Case ordering matters; a broad pattern can make a later case unreachable. Handle null deliberately.

PAUSE & PREDICT

What happens if value is null in this example?

Check your thinking: It returns "missing" because case null is explicit.

REMEMBER THIS

Pattern matching for switch and record patterns became final in Java 21.

JAVA 21 / VISUAL LESSON

Record patterns: unpack data

01 WHY WOULD WE NEED THIS?

You receive a record and want to read its components while checking its type.



02 BEFORE: HOW WE DID IT

```
if (obj instanceof Point) {
 Point p = (Point) obj;
 System.out.println(p.x() + p.y());
}
```

03 AFTER: THE NEW WAY

```
record Point(int x, int y) {}
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + y);
}
```

WHAT IMPROVED?

Record patterns combine checking and unpacking record components.

WATCH OUT

Patterns depend on record shape. Keep complex nested patterns readable.

PAUSE & PREDICT

If obj is not a Point, are x and y created?

Check your thinking: No. The body runs only on a successful match.

REMEMBER THIS

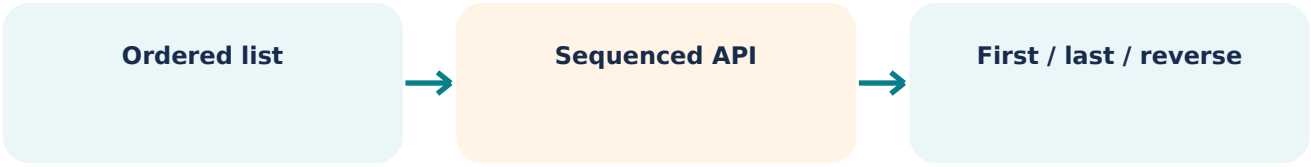
Use records for data; record patterns for convenient deconstruction.

JAVA 21 / VISUAL LESSON

Sequenced collections: first and last

01 WHY WOULD WE NEED THIS?

You want a standard way to get the first or last element of an ordered collection.



02 BEFORE: HOW WE DID IT

```
String first = list.get(0);
String last = list.get(list.size() - 1);
```

03 AFTER: THE NEW WAY

```
String first = list.getFirst();
String last = list.getLast();
List<String> reverseView = list.reversed();
```

WHAT IMPROVED?

A shared API for ordered collections, including first, last, and reversed views.

WATCH OUT

reversed() is a view, not necessarily a copy. Empty collections throw on getFirst/getLast.

PAUSE & PREDICT

Does reversed() always allocate a new independent list?

Check your thinking: No. It returns a reverse-ordered view.

REMEMBER THIS

Java 21 added SequencedCollection, SequencedSet, and SequencedMap.

Your master comparison sheet

Editable list	new ArrayList<>()	Callers can mutate shared state
Fixed-size array view	Arrays.asList(array)	set works; add/remove fail
Live read-only view	Collections.unmodifiableList(x)	Changes to x are visible
Read-only snapshot	List.copyOf(x)	Shallow; rejects null
Fixed known elements	List.of(a, b)	Unmodifiable; rejects null
Readable local type	var	Type inferred at compile time
Simple DTO	record	Mutable components still mutable
I/O-heavy concurrency	Virtual threads	Not a CPU speed trick

Final challenge: reason before running

```
List<String> original = new ArrayList<>(List.of("Java"));
List<String> live = Collections.unmodifiableList(original);
List<String> snapshot = List.copyOf(original);
original.add("Python");
System.out.println(live);
System.out.println(snapshot);
```

ANSWER

live prints [Java, Python]. snapshot prints [Java]. The first is a view of the original list; the second is a snapshot of element references.

Next practice project

Build a tiny payment-service console app. Start with Java 8-style DTOs and collections. Then refactor to List.of, records, switch expressions, pattern matching, and a virtual-thread executor for simulated I/O tasks. After each change, write down: What got simpler? What behavior changed? What could break?

Reference: Official Java release notes and JEP index: docs.oracle.com/en/java/ and openjdk.org/jeps/0. Release attribution in this workbook distinguishes preview from final features.